

Operations on Polynomials

Evaluation

Given a polynomial $A(x)$ and a number x_0 , compute $A(x_0)$.

Can we do this in $O(n)$ time?

Evaluation

Given a polynomial $A(x)$ and a number x_0 , compute $A(x_0)$.

Can we do this in $O(n)$ time?

Yes! This can be done in $O(n)$ time using $O(n)$ arithmetic operations via Horner's rule.

Evaluation

Given a polynomial $A(x)$ and a number x_0 , compute $A(x_0)$.

Can we do this in $O(n)$ time?

Horner's Rule: $A(x) = a_0 + x(a_1 + x(a_2 + \dots x(a_{n-1}) \dots))$.

At each step, a sum is evaluated, then multiplied by x , before beginning the next step. Thus $O(n)$ multiplications and $O(n)$ additions are required.

Addition

Given two polynomials $A(x)$ and $B(x)$,

compute $C(x) = A(x) + B(x) (\forall x)$.

This takes $O(n)$ time using basic arithmetic, because

$$c_k = a_k + b_k.$$

Polynomial multiplication

- Input: degree d polynomials P and Q
- $P(x) = p_0 + p_1 x + p_2 x^2 + \cdots + p_d x^d.$
- $Q(x) = q_0 + q_1 x + q_2 x^2 + \cdots + q_d x^d.$
- $P(x) Q(x) = p_0 + (p_0 q_1 + p_1 q_0)x + \cdots + p_d q_d x^{2d}.$

Polynomial multiplication

- Input: degree d polynomials P and Q
- $P(x) = p_0 + p_1 x + p_2 x^2 + \cdots + p_d x^d.$
- $Q(x) = q_0 + q_1 x + q_2 x^2 + \cdots + q_d x^d.$
- $P(x) Q(x) = p_0 + (p_0 q_1 + p_1 q_0)x + \cdots + p_d q_d x^{2d}.$
- Assume integer multiplication in unit time.
 - School multiplication $O(d^2)$ time.
- Can we do better?

Polynomial multiplication

- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
- $B(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_{d-1} x^{d-1}$
- $A(x)B(x)$
 $= a_0 b_0 + (a_0 b_1 + a_1 b_0) x + (a_0 b_2 + a_1 b_1 + a_2 b_0) x^2$
 $+ \cdots + a_{d-1} b_{d-1} x^{2d-2}$

$$\bullet \quad A(x)B(x) = \sum_{j=0}^{2d-2} x^j \left(\sum_{i=0}^j a_i b_{j-i} \right)$$

Polynomial multiplication

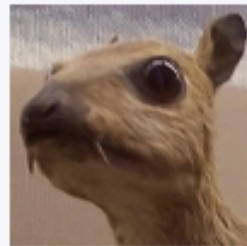
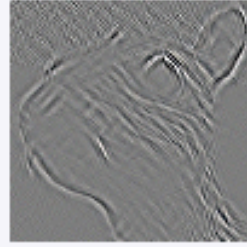
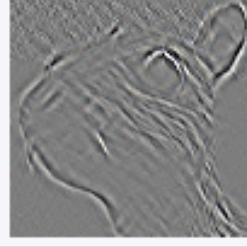
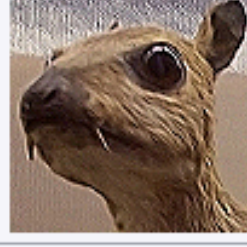
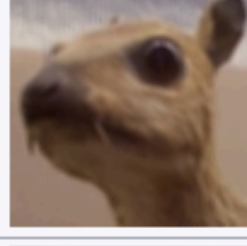
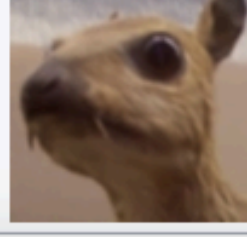
- $A(x)B(x)$
 $= a_0 b_0 + (a_0 b_1 + a_1 b_0) x + (a_0 b_2 + a_1 b_1 + a_2 b_0) x^2$
 $+ \cdots + a_{d-1} b_{d-1} x^{2d-2}$
- Assuming unit cost arithmetic operations, what is the time complexity?
- Naive algorithm $O(d^2)$
- Can we use Karatsuba's idea for polynomial multiplication?

Convolution

- Typically, thought as an operation on vectors...
 - $a = (a_0, a_1, a_2, \dots, a_{m-1}) \in R^m$
 - $b = (b_0, b_1, b_2, \dots, b_{n-1}) \in R^n$
- $a * b = (a_0 b_0, (a_0 b_1 + a_1 b_0), (a_0 b_2 + a_1 b_1 + a_2 b_0),$
 $\dots, (\sum_i a_i b_{j-i}), \dots, a_{m-1} b_{n-1}) \in R^{m+n-1}$
- Dot product with a sliding window

Applications of Convolution

- Signal processing
 - Smoothing of noisy data
 - Covid cases per day: take seven day averages
- Image processing: 2D convolution (bivariate polynomial multiplication)
- Convolutional neural networks

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 x 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

- Source: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))

Applications of Convolution

- Probability distribution of a sum of two random variables

- Dice with

Outcome	1	2	3	4	5	6
Probability	0.2	0.3	0.1	0.1	0.2	0.1

- Sum of two such dice

Outcome	1	2	3	4	5	6	7	...
Probability	0	0.04	0.12					

Polynomial multiplication / Convolution

- Can we get faster than the naive $O(d^2)$ algorithm?
- Different representations of a polynomial
 - coefficients
 - roots
 - evaluations
- Claim: given d evaluations, there is a unique degree $d-1$ polynomial satisfying those evaluations.

Unique polynomial with d evaluations

- **Claim:** given d evaluations, there is a unique degree $d-1$ polynomial satisfying those evaluations.
- **Proof:**
 - Suppose there are two different degree $d-1$ polynomials $P(x)$ and $Q(x)$ which have the same d evaluations.
 - Define $R(x) = P(x) - Q(x)$.
 - Then $R(x)$ is zero at these d points
 - But, a degree $d-1$ polynomial cannot have d roots.
Contradiction.

Roots of a polynomial

- **Claim:** A (nonzero) degree $d-1$ polynomial can have at most $d-1$ roots
- **Proof:** We prove it by induction.
- Base case: degree 1 polynomial has exactly one root.
- Induction hypothesis: a degree $d-2$ polynomial has at most $d-2$ roots
- Induction step: Let $P(x)$ be a polynomial of degree $d-1$.
 - Let α be a root of $P(x)$. Then $x-\alpha$ divides $P(x)$.
 - Let $P(x) = (x-\alpha) Q(x)$. If $P(\beta) = 0$ for some $\beta \neq \alpha$, then $Q(\beta) = 0$.
 - Hence, all other roots of $P(x)$ are also roots of $Q(x)$.
 - By induction hypothesis, $Q(x)$ has at most $d-2$ roots.
 - Hence, $P(x)$ has at most $d-2+1 = d-1$ roots.

Polynomial Representations

- Given d coefficients, how much time needed to compute evaluations of the polynomial at d points?
- Naively, each evaluation will need $O(d)$ multiplications and additions
- Overall time $O(d^2)$
- Can evaluation at one point help with evaluating at another point?

Polynomial representations

- Computations with different representations

	Addition	Multiplication
Coefficients	$O(d)$	$O(d^2)$
Roots	?	$O(d)$
Evaluations	$O(d)$	$O(d)$

Metrix View

- $V \cdot A =$
$$\begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^d \\ 1 & x_2 & x_2^2 & \cdots & x_2^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^d \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Vandermonde matrix V , with entires - $v_{jk} = x_j^k$

But, it takes quadratic time...

Polynomial representations

- Computations with different representations

	Addition	Multiplication
Coefficients	$O(d)$	$O(d^2)$
Roots	?	$O(d)$
Evaluations	$O(d)$	$O(d)$

- But, can we convert between different representations?

Polynomial representations

- Computations with different representations

	Addition	Multiplication
Coefficients	$O(d)$	$O(d^2)$
Roots	?	$O(d)$
Evaluations	$O(d)$	$O(d)$

- Idea - Combine best of each representations...
 - By converting coefficients to evaluations ... then convert back!

Polynomial representations

- Computations with different representations

	Addition	Multiplication
Coefficients	$O(d)$	$O(d^2)$
Roots	?	$O(d)$
Evaluations	$O(d)$	$O(d)$

- Why not do everything with evaluation? — not easy, quadratic!

Coefficients to evaluations

- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
- $A(1) = a_0 + a_1 + a_2 + \cdots + a_{d-1}$
- $A(-1) = a_0 - a_1 + a_2 - \cdots - a_{d-1}$
- Total additions = $2d-2$

Coefficients to evaluations

- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
- $A(1) = a_0 + a_1 + a_2 + \cdots + a_{d-1}$
- $A(-1) = a_0 - a_1 + a_2 - \cdots - a_{d-1}$
- Total additions = $2d-2$
- First compute
even-sum = $a_0 + a_2 + a_4 + \cdots + a_{d-2}$
odd-sum = $a_1 + a_3 + a_5 + \cdots + a_{d-1}$

Coefficients to evaluations

- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
- $A(1) = a_0 + a_1 + a_2 + \cdots + a_{d-1}$
- $A(-1) = a_0 - a_1 + a_2 - \cdots - a_{d-1}$
- Total additions = $2d-2$
- First compute
even-sum = $a_0 + a_2 + a_4 + \cdots + a_{d-2}$
odd-sum = $a_1 + a_3 + a_5 + \cdots + a_{d-1}$
- $A(1) = \text{even-sum} + \text{odd-sum}$
- $A(-1) = \text{even-sum} - \text{odd-sum}$
- Total additions = d

Coefficients to evaluations

- Similar trick with $A(\alpha)$ and $A(-\alpha)$
- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
- $A_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \cdots + a_{d-2} x^{d/2-1}$
- $A_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \cdots + a_{d-1} x^{d/2-1}$
- $A(\alpha) = A_{\text{even}}(\alpha^2) + \alpha \cdot A_{\text{odd}}(\alpha^2)$
- $A(-\alpha) = A_{\text{even}}(\alpha^2) - \alpha \cdot A_{\text{odd}}(\alpha^2)$
- **Two** evaluations of degree $d-1$ polynomial
→ **Two** evaluations degree $d/2-1$ polynomials

Coefficients to evaluations

- Similar trick with $A(\alpha)$ and $A(-\alpha)$
- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
- $A_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \cdots + a_{d-2} x^{d/2-1}$
- $A_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \cdots + a_{d-1} x^{d/2-1}$
- $A(\alpha) = A_{\text{even}}(\alpha^2) + \alpha \cdot A_{\text{odd}}(\alpha^2)$
- $A(-\alpha) = A_{\text{even}}(\alpha^2) - \alpha \cdot A_{\text{odd}}(\alpha^2)$
- **Two** evaluations of degree $d-1$ polynomial
→ **Two** evaluations degree $d/2-1$ polynomials

Divide & Conquer!

How much did we improve?

- If we calculate it naively then it's still quadratic... because the recursion keeps n at each step.
- Collapsing sets: (courtesy to Jeff Erickson's notes)
 - Either $|X|=1$ (base case) or $|X^2| = |X|/2...$ then X^2 is (recursively) collapsing. Then the recurrence is of the form...

$$\bullet T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n) = O(n \log n)$$

Coefficients to evaluations

- $A(\alpha) = A_{\text{even}}(\alpha^2) + \alpha \cdot A_{\text{odd}}(\alpha^2)$
- $A(-\alpha) = A_{\text{even}}(\alpha^2) - \alpha \cdot A_{\text{odd}}(\alpha^2)$
- Evaluations of $A(x)$ at $\alpha, -\alpha, \beta, -\beta, \gamma, -\gamma, \dots$
- $\rightarrow$ Evaluations of $A_{\text{even}}(x)$ and $A_{\text{odd}}(x)$ at $\alpha^2, \beta^2, \gamma^2, \dots$
- Total work reduced by half
- Can we further apply this trick?
- We will need $\alpha^2 = -\beta^2$?

Roots of unity

- Evaluations of $A(x)$ at $1, -1, i, -i$
- $\rightarrow$ Evaluations of $A_{\text{even}}(x)$ and $A_{\text{odd}}(x)$ at $1, i^2 = -1$
- $\rightarrow$ Evaluations of $A_{\text{even-even}}(x), A_{\text{even-odd}}(x), A_{\text{odd-even}}(x), A_{\text{odd-odd}}(x)$ at 1
- To generalize assume $d = 2^t$
- We will start with d -th roots of unity.

Roots of unity

- $a + ib = r e^{i\theta}$
- $r = \sqrt{a^2 + b^2}$ is the absolute value
- $\theta = \tan^{-1}(b/a)$ is the angle with real axis
- $e^{i\pi} = -1$
- Primitive d -th root of unity = $\omega = e^{2\pi i / d}$
 - All d -th roots of unity = $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$

Properties of roots of unity

- Primitive d -th root of unity = $\omega = e^{2\pi i / d}$
 - $\omega^{d/2} = e^{2\pi i / 2} = -1$
 - All d -th roots of unity = $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$
 - All $(d/2)$ -th roots of unity = $1, \omega^2, \omega^4, \omega^6, \dots, \omega^{2d-2}$
 - $1 + \omega + \omega^2 + \omega^3 + \dots + \omega^{d-1} = 0$
 - $1 + \omega^j + \omega^{2j} + \omega^{3j} + \dots + \omega^{(d-1)j} = 0$ for $0 < j < d$

Discrete Fourier Transform

- Evaluate a degree $d-1$ polynomial at d -th roots of unity

- Evaluations of $A(x)$ at $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$ (d -th roots)

- Given $a_0, a_1, a_2, \dots, a_{d-1}$

- Output

$$a_0 + a_1 \cdot 1 + a_2 \cdot 1^2 + \dots + a_{d-1} \cdot 1^{d-1}$$

$$a_0 + a_1 \omega + a_2 \omega^2 + \dots + a_{d-1} \omega^{d-1}$$

$$a_0 + a_1 \omega^2 + a_2 \omega^4 + \dots + a_{d-1} \omega^{2d-2}$$

$$a_0 + a_1 \omega^{d-1} + a_2 \omega^{2d-2} + \dots + a_{d-1} \omega^{2d-1}$$

Fourier Transform

- $F(k) = \int_{-\infty}^{\infty} f(x) \sin(2\pi kx) dx$

- $G(k) = \int_{-\infty}^{\infty} f(x) \cos(2\pi kx) dx$

- $G(k) + iF(k) = \int_{-\infty}^{\infty} f(x) e^{2\pi kx} dx$

Fast Fourier Transform

- Evaluations of $A(x)$ at $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$
 - Assume d is a power of 2
 - $A_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \dots + a_{d-2} x^{d/2-1}$
 - $A_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \dots + a_{d-1} x^{d/2-1}$
 - $-1 = \omega^{d/2}$
 - $-\omega = \omega^{1+d/2}$
 -
 - $-\omega^{d/2-1} = \omega^{d/2+d/2-1} = \omega^{d-1}$
 - Squares of the d -th roots : $1, \omega^2, \omega^4, \dots, \omega^{2(d/2-1)}$ ($d/2$ -th roots)
- $A(\alpha) = A_{\text{even}}(\alpha^2) + \alpha \cdot A_{\text{odd}}(\alpha^2)$
 - $A(-\alpha) = A_{\text{even}}(\alpha^2) - \alpha \cdot A_{\text{odd}}(\alpha^2)$

Fast Fourier Transform

Cooley-Tukey 1965

- Evaluations of $A(x)$
at $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$ (d -th roots)
- Recursively evaluate $A_{\text{even}}(x)$ and $A_{\text{odd}}(x)$
at $1, \omega^2, \omega^4, \omega^6, \dots, \omega^{d-2}$ ($d/2$ -th roots)
- For $0 \leq k \leq d/2-1$
 - $A(\omega^k) = A_{\text{even}}(\omega^{2k}) + \omega^k \cdot A_{\text{odd}}(\omega^{2k})$
 - $A(\omega^{k+d/2}) = A_{\text{even}}(\omega^{2k}) - \omega^k \cdot A_{\text{odd}}(\omega^{2k})$

Fast Fourier Transform

- Let $T(d)$ be the time to evaluate a degree $d-1$ polynomial at d -th roots of unity.
- For $0 \leq k \leq d/2-1$
 - $A(\omega^k) = A_{\text{even}}(\omega^{2k}) + \omega^k \cdot A_{\text{odd}}(\omega^{2k})$
 - $A(\omega^{k+d/2}) = A_{\text{even}}(\omega^{2k}) - \omega^k \cdot A_{\text{odd}}(\omega^{2k})$
- $T(d) = 2 T(d/2) + O(d)$
- $T(d) = O(d \log d)$

Polynomial multiplication / convolution

- Coefficients $\rightarrow$ Evaluations (FFT)
 - $O(d \log d)$
- Pointwise multiply evaluations of the two polynomials
 - $O(d)$
- Evaluations $\rightarrow$ Coefficients (inverse FFT)
 - Inverse FFT is just FFT with ω replaced by ω^{-1}
 - $O(d \log d)$

Inverse FFT

- Inverse FFT is just FFT with ω replaced by $\omega^{-1} = \omega^{d-1}$

$$e_0 = a_0 + a_1 1 + a_2 1^2 + \cdots + a_{d-1} 1^{d-1}$$

$$e_1 = a_0 + a_1 \omega + a_2 \omega^2 + \cdots + a_{d-1} \omega^{d-1}$$

.....

$$e_{d-1} = a_0 + a_1 \omega^{d-1} + a_2 \omega^{2d-2} + \cdots + a_{d-1} \omega^{2d-1}$$

$$d a_0 = e_0 + e_1 1 + e_2 1^2 + \cdots + e_{d-1} 1^{d-1}$$

$$d a_1 = e_0 + e_1 \omega^{-1} + e_2 \omega^{-2} + \cdots + e_{d-1} \omega^{-d+1}$$

.....

$$d a_{d-1} = e_0 + e_1 \omega^{-d+1} + e_2 \omega^{-2d+2} + \cdots + e_{d-1} \omega^{-2d+1}$$

FFT Implementation issues

- Can we really do additions / multiplications with roots of unity in constant time?
 - approximations: how many bits sufficient?
 - Requires careful implementation.
 - modular arithmetic: work modulo a large enough prime, such that d -th roots of unity exist.
- Is it fast in practice?
 - Iterative implementation, which is memory efficient
 - FFT convolution faster than direct convolution for $d=128$

Fast integer multiplication

- Split the integers into d parts each having t bits
- $a = a_0 + a_1 2^t + a_2 2^{2t} + \cdots + a_{d-1} 2^{(d-1)t}$
- $b = b_0 + b_1 2^t + b_2 2^{2t} + \cdots + b_{d-1} 2^{(d-1)t}$
- Define
 - $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
 - $B(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_{d-1} x^{d-1}$
- Multiply as polynomials (using FFT) and put $x = 2^t$